Working with JS libraries in a typed codebase

COMPLETE LEARNING GUIDE

What You'll Learn

The vast majority of real TypeScript projects depend on JavaScript libraries that were never written with TypeScript in mind, so a practical TypeScript developer needs to know how to bring type safety to code you don't control, not just code you write from scratch. This sub-module covers how @types packages (the DefinitelyTyped project) provide types for untyped JavaScript libraries, writing your own minimal .d.ts declaration files when no types exist anywhere, the declare module escape hatch for quickly unblocking an untyped import, typing callback-heavy and loosely-typed third-party APIs defensively, module augmentation for extending a library's own types, and the ESM/CommonJS interop settings that cause a disproportionate share of real-world 'why won't this import compile' confusion.

1. Why Type Definitions Matter for JS Libraries

A JavaScript library ships no type information at all by default — every import from it would be implicitly typed `any` without a separate source of type declarations, silently disabling type checking for anything that touches it, which defeats much of the value of using TypeScript in the first place. Type declaration (.d.ts) files solve this by describing a library's shape — its exported functions, classes, and types — without containing any actual runtime implementation, letting the compiler check your usage of the library against that description.

2. Installing and Using @types Packages

The DefinitelyTyped project maintains community-written type declarations for thousands of popular untyped JavaScript packages, published under the @types npm scope — installing @types/lodash alongside lodash itself, for example, gives the compiler full type information for every lodash function with no further configuration needed. Many modern libraries now ship their own types directly in their main package (look for an included index.d.ts or a "types" field in package.json), making a separate @types install unnecessary — checking for bundled types first avoids installing a redundant or out-of-sync external declaration package.

- npm install -D @types/lodash installs just the type declarations as a dev dependency — no runtime code is added to the bundle.
- If a package's own package.json already lists a "types" or "typings" field, it ships its own types and a separate @types package either doesn't exist or isn't needed.

```bash
npm install lodash
npm install -D @types/lodash
```

3. Writing Your Own .d.ts Declaration Files

When a library has no bundled types and no @types package exists, a minimal hand-written declaration file fills the gap — it only needs to describe the shape actually used, not the library's entire surface area, and can be expanded incrementally as more of the library gets used. Declaration files use declare to describe something that exists at runtime without providing an implementation (since the real JavaScript file already supplies that), and are typically placed in a types/ directory included via the tsconfig's include or typeRoots settings.

```typescript
// types/some-untyped-lib.d.ts
declare module "some-untyped-lib" {
  export function formatCurrency(amount: number, currencyCode: string): string;
  export interface FormatOptions {
    locale?: string;
    decimals?: number;
  }
  export function format(amount: number, options?: FormatOptions): string;
}
```

4. The declare module Escape Hatch

For a quick, temporary unblock — or a library you genuinely don't want to type properly right now — declare module "library-name"; with no body at all tells the compiler 'trust me, this module exists,' and every import from it is typed as `any`. This restores compilation immediately but reintroduces the exact type-safety gap TypeScript exists to close for every usage of that import, so it should be treated as a stopgap to replace with real types later, not a long-term solution for libraries used extensively throughout a codebase.

```typescript
// types/shims.d.ts — quick unblock, everything from this import becomes `any`
declare module "legacy-untyped-widget";

import Widget from "legacy-untyped-widget"; // typed as `any` — no safety, but it compiles
```

5. Typing Third-Party Callback-Heavy APIs Defensively

Older JavaScript libraries frequently expose loosely-typed callback-based APIs (a callback receiving (err, result) with no strong typing on either), and even when @types exist for them, the types are sometimes wider than the library's actual guaranteed behavior. Wrapping such an API in a small, strongly-typed adapter function — written once, used everywhere — both narrows the type to what your code actually needs and gives you a single place to update if the library's real behavior turns out to differ from its declared types.

```typescript
// A loosely-typed legacy callback API, wrapped in a strongly-typed Promise adapter
function readFileAsync(path: string): Promise<string> {
  return new Promise((resolve, reject) => {
    legacyReadFile(path, (err: Error | null, data?: string) => {
      if (err) reject(err);
      else resolve(data ?? "");
    });
  });
}
```

6. Module Augmentation

Sometimes a library's own types are correct but incomplete for your use case — a plugin adds a new method to a class the library defines, and you want the compiler to know about it. Module augmentation lets you reopen an existing module's declared types and add to them, using the same declare module syntax pointed at the library's actual module name rather than a new one, combined with interface declaration merging for the specific exported type being extended.

```typescript
// Augmenting Express's Request type to add a custom `user` property
import "express";

declare module "express" {
  interface Request {
    user?: { id: number; role: string };
  }
}

// Now, anywhere Express's Request type is used, req.user is recognized:
// app.get("/profile", (req, res) => { console.log(req.user?.id); });
```

7. Handling any From Untyped Dependencies Safely

Even with declaration files in place, some values crossing a library boundary inevitably arrive as `any` (parsed JSON responses, dynamically-shaped config) — the practical discipline is to narrow that `any` to a specific type as early as possible after it enters your code, ideally at the exact boundary where it's received, rather than letting it propagate untyped through several layers of your own application code where it silently disables checking everywhere it touches.

- Treat the boundary where external/untyped data enters your code as the single place to validate and narrow it — after that point, the rest of your code should work with a real, specific type.
- A lightweight runtime validation library (zod, io-ts) can both validate and derive a precise static type from untyped data in one step, which is more robust than a manual type assertion.

8. ESM vs. CommonJS Interop Settings

A large share of confusing 'cannot find module' or 'this import has no default export' errors come from a mismatch between how a library was published (CommonJS's module.exports vs. ECMAScript Modules' export default) and the project's tsconfig interop settings. Enabling esModuleInterop allows the simpler import defaultExport from "cjs-library" syntax to work correctly against CommonJS libraries that don't use a true default export, and allowSyntheticDefaultImports (usually implied by esModuleInterop) silences the type-checking-only complaint about that same pattern — both should essentially always be enabled in a modern tsconfig.json targeting Node.js or bundler-based projects.

```JSON
{
  "compilerOptions": {
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "moduleResolution": "bundler",
    "strict": true
  }
}
```

Common Pitfalls

- Not checking whether a library already bundles its own types before installing a possibly redundant or outdated separate @types package.
- Leaving a quick `declare module "lib";` shim in place long-term, silently keeping every usage of that library typed as `any` indefinitely.
- Letting an `any` value from an untyped dependency propagate deep into application code instead of narrowing it to a real type at the boundary where it's received.
- Forgetting esModuleInterop, then fighting confusing default-import errors against CommonJS libraries that don't export a true ES default.
- Writing an overly broad hand-rolled .d.ts file that describes far more of a library's surface than is actually used, creating maintenance burden for types that may not even be correct.
- Trusting a community @types package's types as infallible when the library's actual runtime behavior has quietly diverged from what the types describe (a common issue with fast-moving libraries).

Key Takeaways

- Check for bundled types (a "types" field in package.json) before installing a separate @types/* package — many modern libraries ship their own.
- declare module "lib"; with no body is a stopgap that types every import from that library as `any` — replace it with real types over time, don't leave it permanently.

- Write only the minimal .d.ts declarations your code actually uses; expand them incrementally rather than trying to type an entire library upfront.
- Narrow `any` values from untyped dependencies to a specific type as close as possible to where they enter your code.
- Module augmentation (declare module pointed at an existing library) is the correct way to add missing properties/methods to a library's own types.
- esModuleInterop and allowSyntheticDefaultImports should be enabled in virtually every modern tsconfig.json to avoid CommonJS/ESM default-import friction.

Learn More

TypeScript Handbook — Modules: Type Declarations
https://www.typescriptlang.org/docs/handbook/modules/theory.html

DefinitelyTyped — @types on npm
https://www.typescriptlang.org/dt/search?search=

TypeScript Handbook — tsconfig: esModuleInterop
https://www.typescriptlang.org/tsconfig/#esModuleInterop